星绽操作系统 dm-crypt/dm-verity

设计与实现技术文档

全国操作系统设计赛——功能挑战赛道

参赛队伍：Go_Go_Go
高校：华南师范大学
成员：张乐桁、陈韵、潘利源

2026 年 8 月 11 日

目录

1 项目概述与背景 **5**

1.1 项目背景 5
1.2 项目目标 5
1.3 双目标架构创新 6
1.4 系统架构概述 6
1.5 文档结构 7
1.6 小结 7

2 系统架构设计 **8**

2.1 设计原则 8
2.1.1 核心抽象：BlockDevice Trait 8
2.1.2 可组合的映射器链模式 9
2.2 核心模块分解 9
2.3 跨环境可移植性 10
2.4 数据流架构 11
2.5 线程安全保证 12
2.6 性能可观测性设计 12
2.7 运行时控制面（Linux 兼容） 13
2.8 小结 13

3 dm-verity 完整性校验详细设计 **14**

3.1 概述 14
3.2 架构组成 14
3.3 默克尔树构建机制 15
3.3.1 构建流程 15
3.3.2 内存与磁盘布局 15
3.4 单块校验路径 16
3.5 只读强制策略 17
3.6 启动参数解析 17
3.6.1 支持的参数 17

3.6.2 解析流程 . . . 17
3.6.3 内核自检机制 . . . 18
3.7 认证后哈希缓存与优化 . . . 18
3.8 校验失败处理 . . . 19
3.9 小结 . . . 19

4 dm-crypt 透明加密详细设计 20
4.1 概述 . . . 20
4.2 系统架构与数据流 . . . 20
4.3 核心组件 . . . 21
4.3.1 DmCryptTable——加密目标配置表 . . . 21
4.3.2 Aes256XtsCipher——核心密码引擎 . . . 21
4.3.3 DmCryptDevice——主设备映射目标 . . . 22
4.4 实现细节：扇区转换与 I/O 路径 . . . 22
4.5 Linux 兼容性验证 . . . 23
4.5.1 兼容性对比 . . . 23
4.5.2 交叉验证方法 . . . 23
4.6 根文件系统加密方案 . . . 23
4.7 小结 . . . 24

5 性能设计与评测 25
5.1 评测原则 . . . 25
5.2 真实性审计 . . . 25
5.3 FIO 结果 . . . 25
5.4 Filebench、SQLite 与启动耗时 . . . 26
5.5 实现侧性能设计 . . . 27
5.6 小结 . . . 27

6 测试与验收 28
6.1 测试体系 . . . 28
6.2 正确性结果 . . . 28
6.3 性能测试结果 . . . 29
6.4 复现命令 . . . 29
6.5 证据可复核性 . . . 30
6.6 小结 . . . 30

7 合规、来源与 AI 声明 31
7.1 概述 . . . 31
7.2 代码与文档来源及开源协议说明 . . . 31

7.2.1 引用的开源代码 . . . 31
7.2.2 开源协议合规性声明 . . . 32
7.3 借鉴基础版本与增量贡献说明 . . . 32
7.3.1 借鉴基础 . . . 32
7.3.2 本队增量贡献 . . . 32
7.4 AI 工具使用声明 . . . 33
7.4.1 使用的 AI 工具及大模型 . . . 33
7.4.2 AI 交互记录 . . . 33
7.4.3 验证机制与责任声明 . . . 36
7.5 Git 提交与开发迭代记录 . . . 36
7.5.1 提交概览 . . . 36
7.5.2 关键迭代节点说明 . . . 38
7.6 小结 . . . 38

8 用户手册与部署指南 39
8.1 概述 . . . 39
8.2 运行环境准备 . . . 39
8.2.1 宿主机要求 . . . 39
8.2.2 依赖工具安装 . . . 40
8.3 快速开始：用户态演示 . . . 40
8.3.1 克隆仓库与运行测试套件 . . . 40
8.3.2 演示透明加密（dm-crypt） . . . 40
8.3.3 演示完整性校验（dm-verity） . . . 41
8.3.4 演示组合栈（dm-crypt + dm-verity） . . . 41
8.3.5 手动命令示例 . . . 41
8.4 星绽内核集成 . . . 42
8.4.1 准备 Asterinas 源码 . . . 42
8.4.2 应用 Device Mapper 补丁 . . . 43
8.4.3 构建内核 . . . 43
8.4.4 构建受保护的根文件系统镜像 . . . 43
8.5 系统运行与参数配置 . . . 44
8.5.1 QEMU 启动命令 . . . 44
8.5.2 内核启动参数配置 . . . 44
8.6 基准测试与性能验证 . . . 45
8.6.1 正式性能测试 . . . 45
8.6.2 启动时间测量 . . . 46
8.6.3 用户态微基准测试 . . . 46
8.7 高级配置与调优 . . . 46

8.7.1 dm-crypt 临时缓冲池调优 46
8.7.2 dm-verity 哈希缓存调优 46
8.7.3 内核阶段统计 47
8.8 故障排查 47
8.8.1 常见问题 47
8.8.2 调试技巧 47
8.9 小结 48

9 研发过程与工程收尾 49
9.1 研发原则 49
9.2 关键工程问题 49
9.2.1 块大小与扇区语义 49
9.2.2 BIO 生命周期 49
9.2.3 verity 信任边界 49
9.3 Asterinas 补丁演进 50
9.4 渐进式保护路径 50
9.5 测试代码收口 51
9.6 已知边界 51
9.7 小结 51

A 附录 A：术语表 52

B 附录 B：dmctl 命令参考 54

Chapter 1

项目概述与背景

1.1 项目背景

星绽操作系统（Asterinas）是一个采用创新性 Framekernel 架构的 Rust 操作系统，通过 OSTD 框架将内核核心组件运行于非特权级，实现了可信计算基（TCB）的最小化。依托 Rust 的所有权与生命周期机制，星绽从根源上规避了内存安全风险，同时在性能关键路径上保持高效执行。

在机密计算（Confidential Computing, CoCo）场景下，星绽计划作为 Guest Kernel 运行于可信虚拟机中。尽管内存数据受 TEE 保护，但持久化存储中的镜像（如 rootfs）仍存储在宿主机的不可信介质上，面临离线篡改、镜像替换、数据窃取等安全威胁。攻击者可借此植入后门或破坏系统完整性，危及整个信任链。

Linux 通过 Device Mapper 提供了 `dm-crypt`（透明数据加密）和 `dm-verity`（只读数据完整性校验）机制 [1, 2]，广泛用于全盘加密与安全启动。本赛题要求在星绽 OS 中，利用 Rust 的内存安全优势，自主实现类 `dm-crypt` 与 `dm-verity` 模块，填补星绽生态在存储安全方面的空白 [6]。

1.2 项目目标

本项目在星绽操作系统上使用 Rust 编程语言实现 `dm-crypt` 与 `dm-verity` 功能，具体技术指标如下：

1. **`dm-verity` 基础功能：**构建 SHA-256 哈希树 [5]，支持内核启动参数配置（如 `rootfs_verity.scheme=dm-verity rootfs_verity.hash=<hash>`），在系统启动时对根文件系统进行完整性验证。
2. **`dm-crypt` 基础功能：**支持透明加解密 I/O 路径，兼容 Linux `aes-xts-plain64` 密码格式 [4]，作为后端支撑 `cryptsetup` 对加密卷的创建和管理 [3]。
3. **根文件系统加密：**使用 `dm-crypt` 加解密根文件系统，确保持久化数据的机密性。
4. **性能与启动要求：**I/O 吞吐及延迟接近 Linux，`dm-verity` 不显著增加内核启动时

间。

5. **技术探索**：探索基于 Rust OS 的可信存储性能优化技术，如认证后哈希缓存、有界临时缓冲池和硬件加速（AES-NI）。

当前版本已完成上述全部指标的用户态验证和星绽内核集成，并包含可审计的补丁序列和 KVM 五轮基准测试证据。

1.3 双目标架构创新

本项目的一个关键创新在于其**双目标架构**：核心逻辑被实现为一个与平台无关的 Rust 库（`dm` crate），该库同时服务于两个目标环境：

- **用户态命令行工具（`dmctl`）**：用于演示、测试和基准测试，能够在宿主机上快速验证加密和完整性机制的正确性。
- **星绽内核集成（`aster-dm`）**：将相同的核心逻辑移植到内核空间，使星绽能够利用 `virtio-blk` 设备作为加密或完整性保护的卷，直接挂载受保护的文件系统（如 ext2）。

这种设计确保了用户态与内核态的逻辑一致性，使得测试和调试更加高效，同时也为后续的性能优化和功能扩展提供了统一的抽象基础。

1.4 系统架构概述

系统的核心抽象是 `BlockDevice` trait，它定义了统一的块级 I/O 接口（`read_block` 和 `write_block`）。不同的存储后端（如内存、文件、物理磁盘）均实现该 trait，而加密和完整性校验模块（`DmCryptDevice`、`DmVerityDevice`）则以“装饰器”模式包装下层设备，形成可堆叠的映射器链。

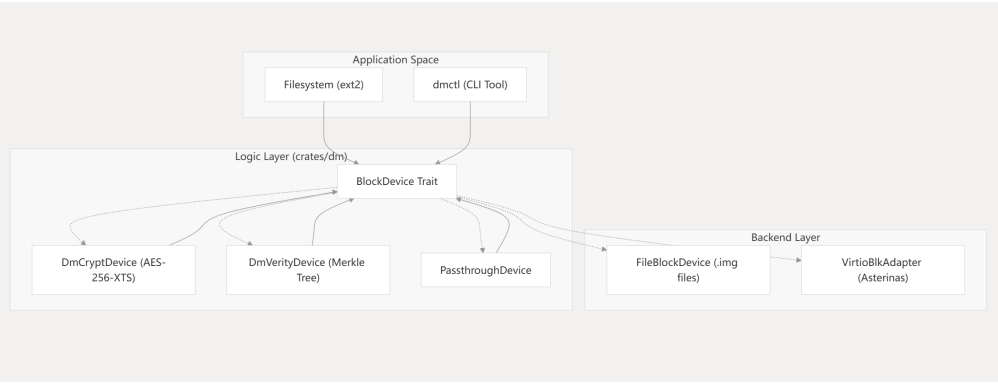


图 1.1: 系统组件关系图

如图所示，文件系统或应用程序与顶层的 `BlockDevice`（例如 `DmCryptDevice`）交互，该设备在执行加密转换后，将 I/O 委托给下一层设备（如 `VirtioBlkAdapter` 或 `FileBlockDevice`），最终到达物理存储介质。

1.5 文档结构

本技术文档共分为九章和两个附录，结构如下：

- **第 2 章系统架构设计**：详细阐述 `BlockDevice` trait、可组合映射器链、模块分解及跨环境可移植性。
- **第 3 章 `dm-verity` 详细设计**：介绍默克尔树构建、单块校验路径、启动参数解析及只读强制策略。
- **第 4 章 `dm-crypt` 详细设计**：说明 `DmCryptTable`、AES-256-XTS 密码引擎、扇区算术及 Linux 兼容性验证。
- **第 5 章性能优化策略**：涵盖认证后哈希缓存、完成 BIO 数据复用、有界临时缓冲池及 AES-NI 硬件加速等优化手段。
- **第 6 章测试与性能分析报告**：包括功能正确性测试（篡改检测、明文/密文验证）、FIO 基准测试、启动开销分析及与 Linux 的交叉验证。
- **第 7 章合规、来源与 AI 声明**：独立章节，严格遵循比赛要求，披露代码来源、开源协议及 AI 工具使用情况。
- **第 8 章用户手册与部署指南**：提供从环境准备到编译、运行、配置的完整操作步骤。
- **第 9 章研发过程与问题解决记录**：记录核心难题（如 Rust 所有权冲突、DMA 路径验证）及解决思路，并展示 Git 迭代历史。
- **附录 A 术语表**：定义代码库中的关键术语。
- **附录 B `dmctl` 命令参考**：列出所有子命令的签名、示例和实现说明。

1.6 小结

本章介绍了项目的背景、目标、双目标架构创新及整体文档结构。后续章节将深入阐述每个模块的设计与实现细节，并通过测试数据验证系统的功能正确性与性能表现。

Chapter 2

系统架构设计

2.1 设计原则

本项目的基本设计原则是使用统一的抽象来表示任何能够进行块级 I/O 的实体。这种抽象允许采用“分层”或“可组合”的架构，使得加密和校验逻辑与底层存储介质完全解耦，从而在用户态命令行工具和星绽内核中实现相同的核心逻辑。

2.1.1 核心抽象：BlockDevice Trait

`BlockDevice` trait 是库中所有存储操作的基础接口。它定义了用于读写固定大小数据块的同步接口。为了支持跨线程的并发访问，并安全地集成到星绽内核或用户态工具中，该 trait 要求实现 `Send` 和 `Sync` 标记。

该 trait 定义如下：

```
pub trait BlockDevice: Send + Sync {
    fn block_size(&self) -> usize;
    fn num_blocks(&self) -> u64;
    fn read_block(&self, block_id: u64, buf: &mut [u8]) -> Result
        <()>;
    fn write_block(&self, block_id: u64, buf: &[u8]) -> Result<()>;
}
```

所有 I/O 方法均返回 `Result<()>`，其中错误被封装在 `DmError` 枚举中。

在 Asterinas 内核中，该 trait 进一步映射到 `aster-block` 组件的 BIO 接口，从而支持真实的 VirtIO 块设备。

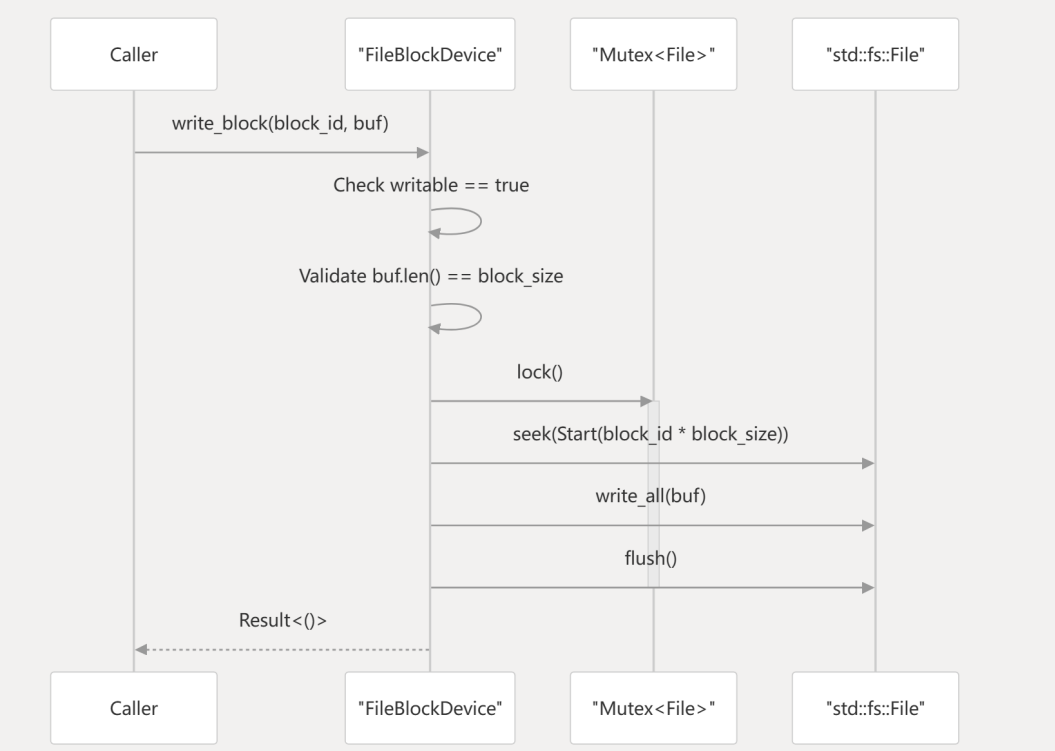


图 2.1: BlockDevice Trait 与后端实现关系图

2.1.2 可组合的映射器链模式

该架构采用类似装饰器（Decorator）的设计模式。其中“映射器”（如 `DmCryptDevice` 和 `DmVerityDevice`）既实现了 `BlockDevice` trait，又在其内部消费（包装）另一个实现了 `BlockDevice` 的对象。这允许功能的任意堆叠与组合。

例如，一个典型的栈可能是：

```
let file = FileBlockDevice::open("disk.img")?;
let crypt = DmCryptDevice::new(file, key)?;
let verity = DmVerityDevice::new(crypt, root_hash)?;
```

在星绽内核中，这种组合通过 `DmDevice` 结构体实现，它根据 `DmTarget` 枚举（Passthrough、Crypt、Verity）分发 BIO 请求。由于每个 mapper 的下层是 `Arc<dyn BlockDevice>`，启动阶段可将 `rootfs_verity.device` 指向已注册的 crypt mapper 名（如 `dm-crypt0`），或显式设置 `dm_stack=crypt,verity`，从而形成与用户态相同的叠栈：`vdc` → `dm-crypt0` → `dm-verity0`。验收脚本为 `tools/run_dm_stack.sh` / 仓库 `scripts/run_dm_stack.sh`。

2.2 核心模块分解

项目被结构化为离散的模块，以清晰地分离密码学原语、数据结构和设备逻辑。

表 2.1: 核心模块职责划分

模块	主要职责	关键实体
block_device	基础 I/O 抽象与物理/内存后端	BlockDevice, FileBlockDevice, MemoryBlockDevice
dm_crypt	使用 AES-XTS 的透明加密逻辑	DmCryptDevice, DmCryptTable
dm_verity	只读完整性校验逻辑	DmVerityDevice
hash_tree	默克尔树的构建与管理	HashTree
sha256	纯 Rust 实现的 SHA-256（无外部依赖）	sha256::sha256
device_mapper	设备链接（chaining）的基础设施	PassthroughDevice, DmStackBuilder
statistics	I/O 性能计数器与阶段归因	InstrumentedDevice, IoStatsSnapshot
runtime	运行时 mapper 管理（Linux dm_ioctl 兼容）	RuntimeMapper, DmControlDevice
error	统一错误处理	DmError, Result

BlockDevice 实现涵盖了多种后端场景：

- MemoryBlockDevice：基于内存向量的设备，主要用于单元测试和易失性存储。
- FileBlockDevice：由宿主机文件系统上的文件支持的持久化设备，由 dmctl 工具使用。
- PassthroughDevice：将调用直接转发给内部设备的包装器，用于验证映射器链。
- VirtioBlkAdapter:（星绽移植版）将内核特定的块设备句柄适配到 BlockDevice 接口，位于 aster-dm 组件中。

2.3 跨环境可移植性

一个核心设计目标是：相同的逻辑同时服务于用户态演示和星绽内核。这是通过严格隔离环境特定代码来实现的。

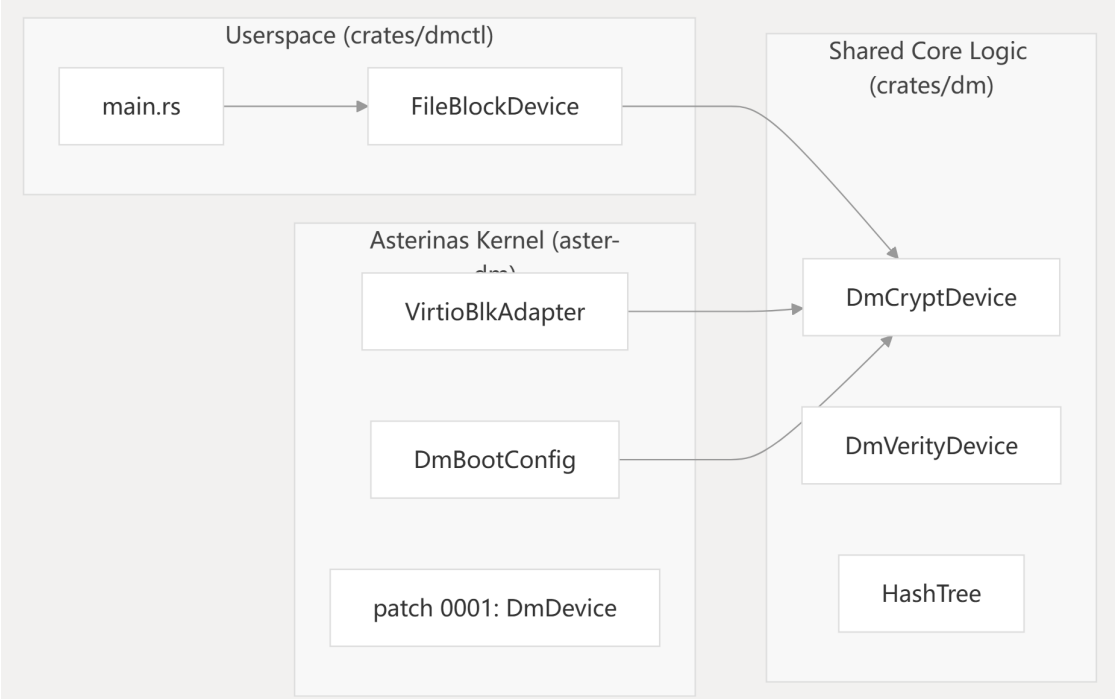


图 2.2: 代码实体映射图（用户态 vs 星绽内核）

用户态：`dmctl` 工具实例化一个 `FileBlockDevice`，并根据命令行参数将其包装在 `DmCryptDevice` 或 `DmVerityDevice` 中。
内核态：在星绽中，`VirtioBlkAdapter` 包装了内核内部的块设备句柄。由于该适配器实现了 `BlockDevice`，`DmCryptDevice` 可以无需任何修改地直接包装它。
`No-Std` 兼容性:核心 `dm` 库设计为最小化依赖（例如,`aes` crate 设置 `default-features = false`），以促进在可能限制标准库的内核环境中执行。

2.4 数据流架构

下图展示了应用程序的 I/O 请求如何穿过该分层栈：

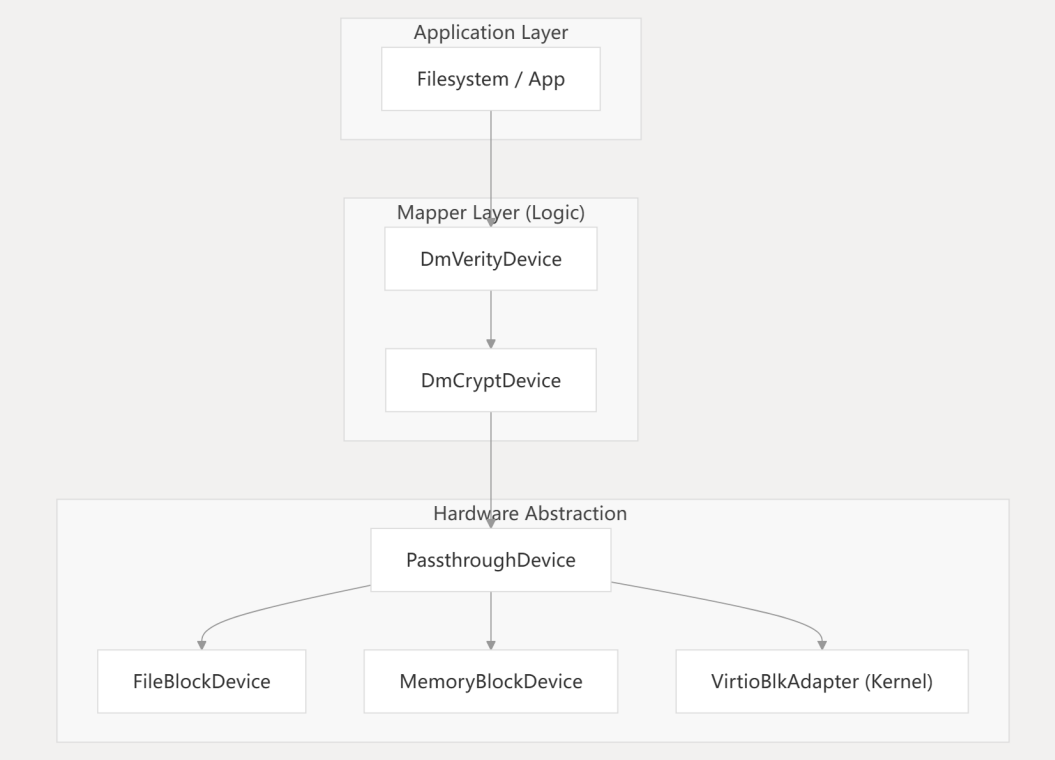


图 2.3: 块 I/O 分层栈数据流图

在此模型中，`DmCryptDevice` 在执行 AES-256-XTS 转换后，将生成的密文传递给链中的下一个设备。类似地，`DmVerityDevice` 拦截读请求，以根据默克尔树校验数据的完整性。

2.5 线程安全保证

所有后端均实现了 `Sync`，这意味着它们可以跨线程共享。

`MemoryBlockDevice`：在整个块向量周围使用 `Mutex`。虽然这确保了安全性，但它将设备上的所有 I/O 串行化。

`FileBlockDevice`：在 `File` 对象周围使用 `Mutex`。这是必要的，因为文件指针（偏移量）是共享状态。锁在 seek、read/write 和 flush 序列的整个持续时间内被持有。

`PassthroughDevice`：依赖于底层设备实现了 `Sync`。

在星绽内核中,`DmDevice` 通过 `enqueue` 方法处理 BIO,该方法内部使用 `spin::Mutex` 保护共享状态，并遵循内核的并发模型。

2.6 性能可观测性设计

为了支持性能归因和优化验证，架构中集成了统计模块：

- `InstrumentedDevice`：一个装饰器，可以插入到任意两层之间，记录读/写次数、字节数、失败次数以及累计和最大延迟（以纳秒计）。
- `IoStatsSnapshot`：提供稳定的快照，便于在基准测试前后对比。
- 在 Asterinas 内核中，通过启动参数 `dm.perf_stats=1` 启用细粒度阶段计时（如 `lower_wait`、`AES-XTS`、`SHA-256` 等），并通过 `/proc/dm-performance-stats` 导出。

  这种设计使得开发者可以精确量化每个映射器层的开销，从而指导优化方向。

2.7 运行时控制面（Linux 兼容）

为了支持 `cryptsetup` 等用户态工具，项目实现了与 Linux `dm_ioctl` 兼容的控制接口：

- `DmControlDevice`:在 `/dev/mapper/control` 提供 ioctl 处理，支持 `DM_DEV_CREATE`、`DM_TABLE_LOAD`、`DM_DEV_SUSPEND`、`DM_DEV_STATUS`、`DM_DEV_REMOVE` 等命令。
- `RuntimeMapper`：管理动态创建的 mapper，包括状态机（Empty、Loaded、Active）和密钥的清除。
- 通过该接口，Asterinas 内核可以接受来自 `cryptsetup` 的标准 table 字符串，动态激活加密映射器，从而实现与 Linux 生态的互操作。

2.8 小结

本章详细阐述了系统的核心架构，包括统一的 `BlockDevice` 抽象、可组合的映射器链模式、模块分解以及跨环境可移植性。通过将核心逻辑与平台细节解耦，并增加性能可观测性和运行时控制面，该项目为后续的加密、完整性校验及性能优化奠定了坚实的基础。在接下来的章节中，我们将深入探讨 `dm-crypt` 和 `dm-verity` 的具体实现细节。

Chapter 3

dm-verity 完整性校验详细设计

3.1 概述

dm-verity 为块设备提供透明的完整性校验。它通过将读取的数据与预计算的默克尔树（Merkle Tree）进行比对，确保从底层存储层读取的数据未被篡改。该实现强制执行只读策略，以在设备的整个生命周期内维持完整性保证。

在星绽操作系统环境中，dm-verity 主要用于保护根文件系统（rootfs）和关键系统镜像，防止攻击者通过离线篡改持久化存储介质植入后门或破坏系统完整性。

3.2 架构组成

完整性校验系统由三个主要组件构成，下图展示了这些组件的映射关系：

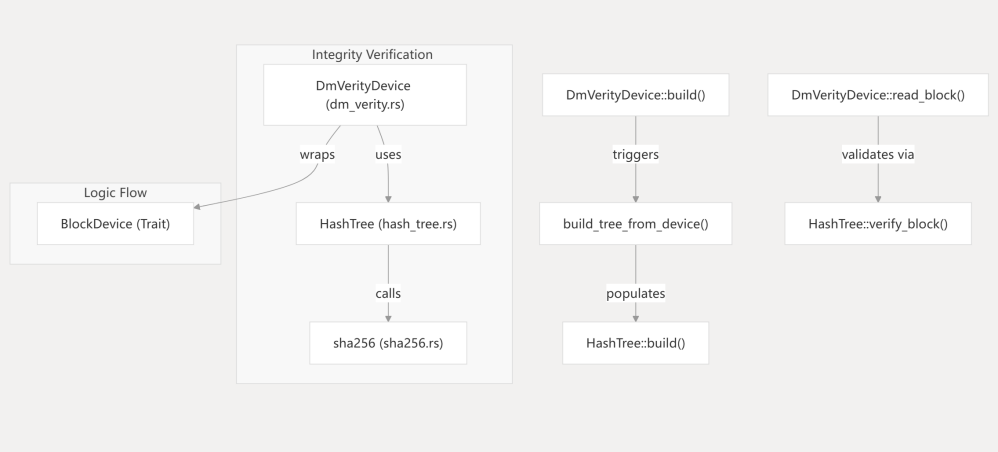


图 3.1: 完整性校验组件映射图

三个核心组件的职责如下：

- `DmVerityDevice`：顶层的 `BlockDevice` 实现，负责拦截读请求并执行校验操作。它是 dm-verity 对外的接口，所有 I/O 请求首先经过该设备。

- `HashTree`：默克尔树结构，用于存储所有数据块和中间节点的 SHA-256 哈希值。该树在镜像构建时生成，在运行时作为校验的参考依据。
- `sha256` 模块：纯 Rust 实现的 SHA-256 哈希算法，用于所有密码学校验和的计算。该实现不依赖任何外部库，确保在受限的内核环境中也能正常运行。

3.3 默克尔树构建机制

3.3.1 构建流程

`HashTree` 通过递归地对块对进行哈希运算来构建，直到得出单一的根哈希（Root Hash）。构建流程如下：

1. **叶子层（Leaf Level）**：使用 `sha256` 对底层设备的每个数据块进行哈希运算，生成叶子节点的哈希值。
2. **中间层（Intermediate Levels）**：将下一层的哈希值两两拼接并进行哈希运算，从而形成上一层。
3. **填充（Padding）**：如果某一层有奇数个哈希值，则最后一个哈希值会被复制一次以凑成一对，确保每层都能配对。
4. **根哈希（Root Hash）**：该过程持续进行，直到只剩下一个哈希值，该值被指定为 root hash，作为整个卷的“信任锚”。

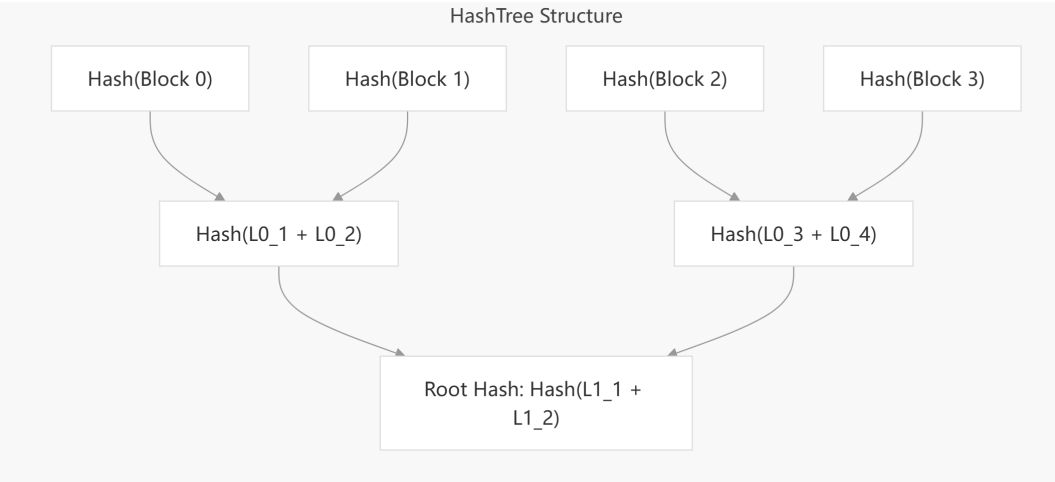


图 3.2: 默克尔树数据流图

3.3.2 内存与磁盘布局

在用户态 `dm` crate 中，默克尔树以 `HashTree` 结构体的形式存储在内存中，该结构体维护一个层级向量，每层存储该层所有节点的哈希值（每个节点 32 字节）。这种设计便于单元测试和快速原型验证。

在星绽内核的 aster-dm 组件中，哈希树采用磁盘布局：数据块从镜像开头连续存放，哈希树从 hash_start_block 开始依次写入各层哈希块。在本项目的默认配置中，哈希树被放置在数据镜像的尾部（hash_start_block = data_blocks），但内核参数也支持自定义起始块号，以适应不同的镜像布局需求。具体参数如下：

- 数据块从块号 0 到 data_blocks - 1 连续存放。
- 哈希树从 hash_start_block 开始，每层哈希块连续存放。
- 块大小固定为 4096 字节，每个哈希块可容纳 4096 / 32 = 128 个 32 字节的 SHA-256 哈希值。
- 每层哈希块的数量由该层需要存储的哈希值数量除以每块可容纳的哈希数（向上取整）决定。
- 最终根哈希块（仅一个块）位于树的最后。

这种设计使得标准 ext2 镜像可以与哈希树共存，而无需修改原有的数据布局，便于在现有系统上部署。

3.4 单块校验路径

当在 DmVerityDevice 上调用 read_block 时，会执行以下校验序列：

1. **数据检索（Data Retrieval）**：从底层块设备读取原始数据块。
2. **路径校验（Path Verification）**：HashTree::verify_block 函数计算检索到的数据的哈希值，并重新计算其到根节点的路径。
3. **兄弟节点检索（Sibling Retrieval）**：在每一层，从存储的 HashTree 中检索“兄弟”节点的哈希值，用于逐级向上计算。
4. **根锚点校验（Root Anchor）**：将重新计算出的哈希值与 DmVerityDevice 结构体中存储的 root hash 进行比对。
5. **强制执行（Enforcement）**：如果哈希值不匹配，或者树的内部根哈希被篡改，则返回 IntegrityViolation 错误，阻止对损坏数据的访问。

表 3.1: 校验路径关键函数职责

函数	职责
DmVerityDevice::read_block	I/O 的入口点；检查树校验结果与根锚点的比对情况。
HashTree::verify_block	迭代地向上计算树的各层哈希，以校验特定的块 ID。
sha256::sha256	计算数据块的 32 字节摘要。

3.5 只读强制策略

为了确保完整性，`DmVerityDevice` 明确禁止写操作。任何调用 `write_block` 的尝试都会导致 `ReadOnlyDevice` 错误。

这种设计反映了 dm-verity 的标准使用场景：哈希树在镜像创建时生成一次，并在运行时仅进行校验。只读策略确保了信任链的完整性，一旦根哈希被验证，整个卷的内容就被锁定，任何写操作都会被拒绝，从而防止攻击者绕过校验机制。

3.6 启动参数解析

为了在系统启动时启用 dm-verity，星绽内核需要从命令行接收配置参数。

3.6.1 支持的参数

表 3.2: dm-verity 启动参数

参数	描述
`rootfs_verity.scheme=dm-verity`	启用 dm-verity 完整性校验标志
`rootfs_verity.device=<设备名>`	底层块设备名称（如 `vdc`）
`rootfs_verity.name=<映射器名>`	创建的 mapper 设备名称（如 `dm-verity0`）
`rootfs_verity.hash=<64-char-hex>`	64 字符十六进制编码的根哈希（32 字节）
`rootfs_verity.data_blocks=<数量>`	数据块总数
`rootfs_verity.hash_start=<块号>`	哈希树在镜像中的起始块号
`rootfs_verity.cache_blocks=<数量>`	已认证哈希缓存块数（默认 256）

3.6.2 解析流程

解析器按空格分割内核命令行字符串，然后逐个匹配上述键。当检测到 `scheme` 参数时，启用 verity 标志。当检测到 `hash` 参数时，将 64 字符的十六进制字符串解码为 32 字节的数组。解码函数执行严格的长度验证（输入必须恰好 64 个字符），并逐字节解析，任何解析失败都会导致配置无效，从而阻止系统挂载受保护的根文件系统。

3.6.3 内核自检机制

在补丁 0002 中，我们引入了自检机制，用于在启动早期验证 dm-verity 路径的正确性。该机制执行以下操作：

1. 通过映射器同步读取第一个块（偏移量 0）。
2. 检查偏移量 1080（1024 + 56）处的 EXT2 魔数值（`0x53EF`）。
3. 如果成功读取到正确的魔数，则证明解密和校验路径功能正常；否则，内核将触发 panic，防止系统在损坏的根文件系统上启动。

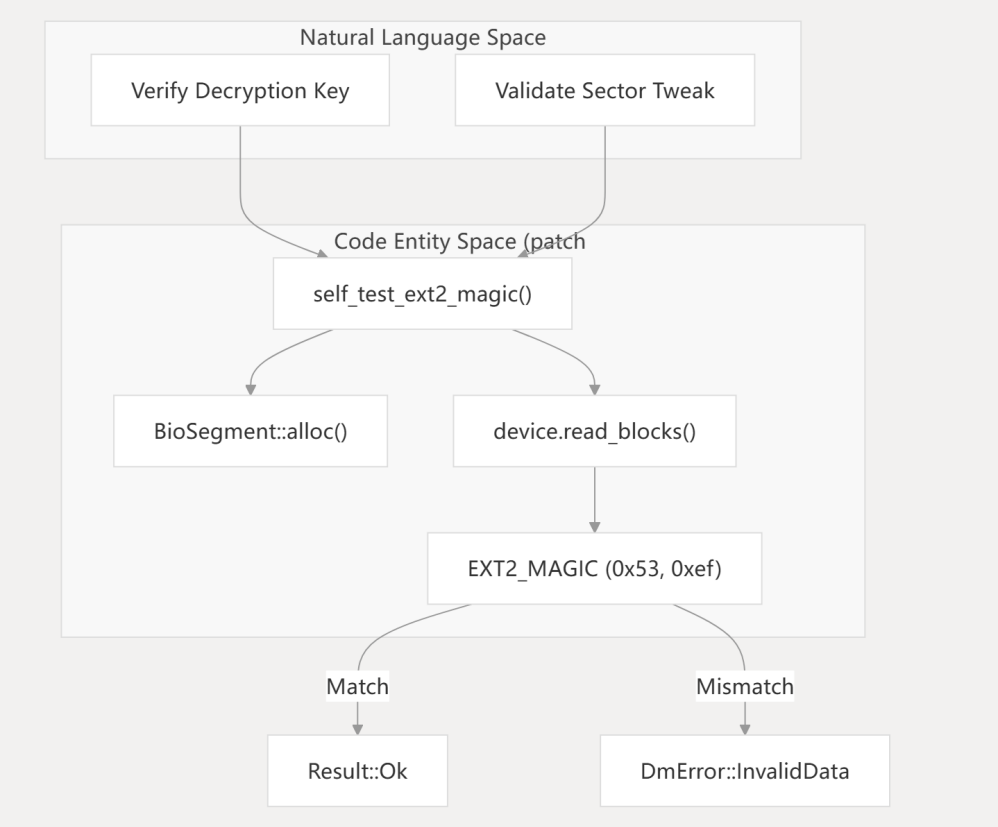


图 3.3: Ext2 超级块验证路径图

在 `aster-dm` 组件中，自检还会记录认证后哈希缓存统计信息（如 `verified`、`hash_hits`、`hash_misses`、`auth_shortcuts`）并在启动日志中输出，便于性能归因。

3.7 认证后哈希缓存与优化

为了减少重复的哈希块读取和计算，`aster-dm` 实现了认证后哈希缓存机制：

- **缓存插入原则**：只有当一个哈希块经过完整的 Merkle 路径验证并成功到达根哈希后，才允许被插入缓存。这保证了缓存中的块始终是可信的。
- **缓存管理**：使用有界 FIFO 队列，容量由启动参数 `rootfs_verity.cache_blocks` 控制（默认 256）。当缓存满时，最早插入的块被淘汰。

- **已认证路径捷径：**当缓存命中时，直接使用缓存的哈希块与当前数据摘要比对，无需重新计算上层哈希后缀。这一优化显著减少了路径验证的计算量，并由计数器 `auth_shortcuts` 记录命中次数。
- **BIO 数据复用：**对于 4 KiB 对齐的读请求，直接使用下层已经完成的 BIO 数据块进行校验，避免为了认证再次读取同一数据块。计数器 `data_rereads` 用于监控是否发生重复读取。

正式 KVM 矩阵已覆盖 7 个只读工况，每项五轮；4 KiB 顺序读中位带宽为 7.505 MiB/s。缓存只接收沿完整路径成功认证的 hash block，因此不会绕过 root hash 信任边界。

3.8 校验失败处理

当 `IntegrityViolation` 错误被触发时，系统根据上下文采取不同的处理策略：

1. `DmVerityDevice` 向调用者返回 I/O 错误。
2. 如果该错误发生在根文件系统访问期间，内核将触发 panic，阻止系统继续执行任何可能依赖损坏数据的操作。
3. 如果发生在普通文件访问中，应用程序将收到 I/O 错误，从而可以优雅地处理异常情况。

在篡改检测验证中，我们通过篡改命令在底层镜像中修改单个字节，然后尝试通过 `DmVerityDevice` 读取该块，确认系统能够正确检测并报告完整性违规。版本化证据显示篡改后启动日志输出 `status=integrity-rejected`，且没有任何 `root switch PASS` 标记。

3.9 小结

本章详细阐述了 dm-verity 的设计与实现，包括默克尔树的构建与校验路径、只读强制策略、启动参数解析、内核自检机制以及认证后哈希缓存等优化措施。通过将哈希树与数据分离存储，并在启动时进行严格的根哈希验证，dm-verity 为星绽操作系统提供了强大的只读完整性保护能力。在下一章中，我们将转向 dm-crypt 的设计与实现，探讨如何为存储设备提供透明加密保护。

Chapter 4

dm-crypt 透明加密详细设计

4.1 概述

dm-crypt 子系统提供透明的块级加密，确保写入物理存储设备的数据被加密，而读回的数据被实时解密。本实现专注于高性能的 AES-256-XTS 加密，并严格兼容 Linux 内核的 dm-crypt 表格式和 plain64 IV（初始化向量）生成机制。

在星绽操作系统中，dm-crypt 主要用于：

- 保护根文件系统（rootfs）的机密性，防止物理介质被窃取后泄露数据。
- 为数据盘提供透明加密，使用户数据在持久化存储中始终保持密文状态。
- 作为 `cryptsetup` 工具的后端支撑，兼容 Linux 生态的加密卷管理流程。

4.2 系统架构与数据流

`DmCryptDevice` 充当任何实现 `BlockDevice` trait 的对象的包装器。它拦截 `read_block` 和 `write_block` 调用，在将数据传递给底层“下层”设备之前或之后执行密码学转换。

数据流的关键步骤如下：

1. 应用程序或文件系统向 `DmCryptDevice` 发起 `write_block` 请求。
2. `DmCryptDevice` 根据逻辑块 ID 和 `iv_offset` 计算对应的物理扇区号。
3. 调用 `Aes256XtsCipher` 对数据进行加密，使用扇区号作为 plain64 IV。
4. 将密文写入底层的 `BlockDevice`（如 `VirtioBlkAdapter` 或 `FileBlockDevice`）。
5. 最终数据以密文形式持久化到物理存储介质。

读路径是上述过程的逆操作：从底层设备读取密文，使用相同的扇区号进行解密，然后返回明文给调用者。

4.3 核心组件

4.3.1 DmCryptTable——加密目标配置表

`DmCryptTable` 结构体表示 crypt 目标的配置，旨在兼容 Linux `dmsetup` 的表行格式。它实现了 `FromStr` trait 以解析标准的空格分隔字符串。

表 4.1: `DmCryptTable` 字段说明

字段	描述
`cipher`	密码算法，固定为 `aes-xts-plain64`
`key`	十六进制编码的加密密钥（AES-256-XTS 为 64 字节）
`iv_offset`	扇区编号的初始向量偏移量
`device`	底层块设备的路径或标识符
`offset`	底层设备上的数据偏移量（以扇区为单位）
`options`	可选参数列表（如 `allow_discards`）

解析器对标准 Linux 表字符串进行标记化，例如：

```
"aes-xts-plain64␣<64-byte-hex-key>␣0␣/dev/vda␣0"
```

支持的可选参数包括 `allow_discards`、`same_cpu_crypt`、`submit_from_crypt_cpus`、`no_read_workqueue`、`no_write_workqueue`、`iv_large_sectors`、`sector_size:512`，实现与 Linux 行为一致。解析器会验证选项计数（第一个选项为选项个数）并识别已声明的选项。

4.3.2 Aes256XtsCipher——核心密码引擎

这是核心密码引擎，使用两个 AES-256 实例实现 XTS 模式：一个用于数据块，另一个用于微调（Tweak/IV）。

密钥要求：严格需要 64 字节（32 字节用于数据密钥，32 字节用于微调密钥）。如果密钥长度不正确，初始化将失败并返回 `DmError::InvalidKey`。

扇区大小：固定为 512 字节，以兼容标准 dm-crypt 扇区单位。这是 Linux 内核 dm-crypt 的约定，保持一致性是实现逐字节兼容的前提。

IV 生成（plain64）：将 64 位扇区号编码到小端序 128 位 IV 的低半部分，然后由微调密钥进行加密。具体流程如下：

1. 将扇区号（`u64`）转换为小端序字节序。
2. 将转换后的 8 字节填入 16 字节缓冲区的低 8 位。
3. 高位清零（plain64 不使用高位）。
4. 微调密钥对完整的 16 字节 IV 进行加密，生成最终的微调值。

5. 数据密钥与微调值配合，执行 XTS 模式的加密/解密操作。

在星绽内核中，该引擎支持 AES-NI 硬件加速：当 CPU 支持 AES-NI 且构建时启用了 `dm_aes_ni` 特性时，使用内联汇编直接调用 AES-NI 指令，并在操作前后保存/恢复 FPU/XMM 状态，确保不被中断上下文破坏。

4.3.3 DmCryptDevice——主设备映射目标

主设备映射目标，负责管理扇区到块的算术运算，并确保 I/O 操作与底层设备的块大小对齐。

扇区算术：由于 `BlockDevice` trait 以块（通常为 4KiB）为单位操作，而 dm-crypt 以 512B 扇区为单位操作，`DmCryptDevice` 需要在两者之间进行转换：

1. 根据逻辑块 ID 和 `iv_offset` 计算起始扇区号。
2. 对于块内的每个扇区，依次调用密码引擎进行加密或解密。
3. 处理扇区偏移与块边界的对齐问题，确保读写操作正确映射。

构造函数（`from_table`）：验证底层设备的块大小是否为 512B 扇区大小的倍数，并计算 `data_offset_blocks`。如果块大小不是 512 的倍数，初始化将失败。同时，构造函数会检查数据偏移量是否超出设备容量，并计算逻辑扇区数。

4.4 实现细节：扇区转换与 I/O 路径

`transform` 函数是基于块的 `BlockDevice` 接口与基于扇区的 `Aes256XtsCipher` 之间的桥梁。

转换逻辑的核心要点：

- 每个 4KiB 块包含 8 个 512B 扇区。
- 加密时，按扇区逐个处理，每个扇区使用其独立的扇区号作为 IV。
- `data_offset_blocks` 用于支持在设备开头预留空间（例如 LUKS 头部）。
- 实际物理块 ID = 逻辑块 ID + `data_offset_blocks`。

I/O 路径的优化措施包括：

- 使用有界临时缓冲池（`scratch_pool`）复用加解密缓冲区，避免热路径中的频繁分配。池的容量由启动参数 `dm_crypt.scratch_pool` 控制（默认 32），设为 0 可关闭池便于 A/B 性能对比。
- 写路径将明文从调用方的 BIO 段复制到临时缓冲区，加密后再通过 `BioSegment::alloc_bytes` 分配下层 I/O 段写入，确保不修改原始调用方数据。
- 读路径从下层读取密文到临时缓冲区，解密后再写回调用方的 DMA 段。

4.5 Linux 兼容性验证

该实现已针对 Linux 内核进行验证，以确保 `aes-xts-plain64` 密码模式的逐字节一致性。

4.5.1 兼容性对比

表 4.2: Linux 兼容性对比

特性	本项目实现	Linux 兼容性
密码算法	AES-256-XTS	`aes-xts-plain64`
IV 生成	plain64（小端序 64 位扇区）	完全一致
密钥大小	512-bit（64 字节）	完全一致
扇区大小	512 Bytes	完全一致
表格式	`DmCryptTable`	兼容 `dmsetup`
可选参数	支持 Linux 已声明的选项	部分兼容

4.5.2 交叉验证方法

我们使用验证脚本进行逐字节兼容性验证：

1. 在 Linux 系统中，使用 `dmsetup` 创建 `aes-xts-plain64` 映射器，写入特定位模式（例如全 0x5a）。
2. 在 Rust 实现中，使用 `dmctl crypt-table-fill` 命令，使用完全相同的密钥和参数，写入相同模式。
3. 使用 `cmp` 命令比较两者生成的密文镜像。
4. 验证通过的标准是：两个文件逐字节完全相同。

版本化证据中已包含固定向量的 SHA-256 校验和（`linux-xts-plain64.sha256`），确保 Rust 实现与 Linux 内核的输出一致。

4.6 根文件系统加密方案

为了实现对根文件系统的加密保护，系统在启动阶段执行以下流程：

1. 内核引导程序传递启动参数：

```
dm_crypt.device=/dev/vda dm_crypt.name=dm-crypt0 \
dm_crypt.key=<128-char-hex-key> dm_crypt.scratch_pool=32
```

2. 星绽内核在初始化阶段解析这些参数，构建配置结构。

3. aster-dm 组件实例化 VirtioBlkAdapter 包装物理设备，然后使用 DmCryptDevice 包装该适配器。
4. 执行 self_test_ext2_magic 自检，验证 AES-XTS 解密路径是否正确（检查 EXT2 魔数 0x53EF）。如果自检失败，内核将 panic。
5. initramfs 脚本（使用 dm-root 模式）挂载加密设备到 /newroot。
6. 执行 pivot_root 将根文件系统切换到加密卷。
7. 系统从加密的根文件系统正常启动，串口日志显示 PASS system root switched to protected mapper，并输出 AES 后端信息（如 backend=AES-NI）。

加密根文件系统还支持 scratch_pool 参数控制临时缓冲池容量，默认 32，可通过 dm_crypt.scratch_pool=0 关闭以进行诊断。真实性审计后的正式 KVM 矩阵覆盖 13 个工况，每项五轮；4 KiB 顺序读中位带宽为 8.908 MiB/s。由于旧 raw 证据与无效 mapper 结果混在同一批次，本报告不再计算 Asterinas 内部 raw 倍率。

4.7 小结

本章详细阐述了 dm-crypt 的设计与实现，包括 DmCryptTable 配置解析、Aes256XtsCipher 密码引擎、扇区算术转换、有界临时缓冲池优化、Linux 兼容性验证以及根文件系统加密方案。通过严格遵循 aes-xts-plain64 标准，并利用交叉验证确保与 Linux 内核的逐字节兼容，dm-crypt 为星绽操作系统提供了可靠的透明加密能力。在下一章中，我们将讨论性能优化策略，包括认证后哈希缓存、完成 BIO 数据复用、有界临时缓冲池和硬件加速等技术手段。

Chapter 5

性能设计与评测

5.1 评测原则

性能评测同时覆盖 Linux host 上的真实块设备对照和 Asterinas KVM 中的保护 mapper。两类环境分别报告，不使用跨平台结果计算倍率。每个有效工况固定运行五轮，保留原始 JSON、CSV、串口日志和环境元数据，并报告均值、中位数和标准差。

FIO 使用的诚实字段如下：

- `iodepth_set` 仅表示 fio 配置值；
- 无法观测真实并发时，`max_in_flight_observed` 记为 `unavailable`；
- dm-verity 是只读目标，写入工况记为 N/A；
- Filebench 使用工具原始 MB/s，FIO 使用 MiB/s，不混用单位。

正式合同见仓库 `docs` 目录,最终结果见 `docs/evidence` 下的 `final-performance-20260811` 目录。

5.2 真实性审计

早期 Asterinas dm-crypt FIO 在 mapper 初始化失败后继续执行，fio 将不存在的设备路径创建为普通文件，产生了数量级异常。该批结果被分类为 `HARNESS_BUG`，未进入正式矩阵，旧数字和失败重试也不作为生成器输入。重新执行的 dm-crypt 使用 guest 内真实 `/dev/mapper/dm-crypt-bench`，dm-verity 则使用修正后的真实 mapper runner。

5.3 FIO 结果

Linux host 对照在同一 backing image 上创建 raw loop block device 和 dm-crypt mapper，统一使用 `libaio`、`direct=1`、`numjobs=1`。代表性五轮中位数如下：

表 5.1: Linux host FIO 五轮中位带宽（MiB/s）

负载	块大小	QD	raw	dm-crypt
read	4 KiB	1	16.296	15.010
read	256 KiB	1	129.599	128.337
randread	4 KiB	16	16.163	15.558
randwrite	256 KiB	1	127.062	117.576

Asterinas KVM 只展示真实保护 mapper 的独立结果，不与 Linux raw 计算倍率：

表 5.2: Asterinas KVM FIO 五轮中位带宽（MiB/s）

负载	块大小	QD	dm-crypt	dm-verity
read	4 KiB	1	8.908	7.505
read	256 KiB	1	97.456	53.363
randread	4 KiB	16	8.614	4.618
randwrite	256 KiB	1	93.541	N/A（只读）

正式矩阵共有 Asterinas dm-crypt 13 项、dm-verity 7 项、Linux raw 13 项和 Linux dm-crypt 13 项，每项五轮；另有 6 个 dm-verity 写入 N/A。

5.4 Filebench、SQLite 与启动耗时

表 5.3: 补充性能测试五轮中位数

测试	raw/plain	dm-crypt	dm-verity
Filebench fileserver（ops/s）	147672.049	156507.741	N/A
Filebench fileserver（MB/s）	3542.7	3752.1	N/A
SQLite 10000 行总耗时（s）	0.060128	0.066395	0.003865（只读）
Asterinas 启动耗时（s）	51.331303	51.629820	52.743415

SQLite dm-verity 只执行查询阶段，不能与完整写事务总耗时横比。启动时间从 QEMU 进程启动计至串口出现受保护根文件系统 PASS 标记；相对 plain 的中位开销为 dm-crypt 0.582%，dm-verity 2.751%。

5.5 实现侧性能设计

dm-crypt 使用 AES-NI 后端和有界临时缓冲池，降低 AES-XTS 与频繁分配的开销；dm-verity 复用已完成 BIO 数据，并对经过完整认证的 hash block 使用有界缓存。缓存遵循“先认证、后插入”，命中不会降低 root hash 信任边界。所有优化仍以 R01–R03、C01–C05 和 V01–V07 正确性测试为前置门禁。

5.6 小结

最终报告只接受能够定位到真实 block device 或 mapper、参数完整且轮数满足合同的数据。完整 46 个 PASS 单元格、6 个 N/A、逐轮统计和证据路径见 `PERFORMANCE_MATRIX.csv` 与中文性能验证报告。

Chapter 6

测试与验收

6.1 测试体系

项目采用用户态单元测试、Linux 兼容性测试、Asterinas KVM 正确性测试和正式性能测试四层门禁。只有真实执行并保留原始证据的项目才标记为 PASS。

表 6.1: 测试层级

层级	验证内容	入口
Rust workspace	加密、哈希树、映射器和命令行	`cargo test --workspace`
Linux 兼容性	AES-XTS plain64 逐字节一致	`verify_linux_xts_compat.sh`
Asterinas KVM	mapper、rootfs、错误密钥和物理损坏	Asterinas tools runner
性能	FIO、Filebench、SQLite 和 boot	`scripts/run_*_matrix.sh`

6.2 正确性结果

固定环境为 Asterinas 官方开发镜像、x86_64、QEMU/KVM、`-cpu host`、1 vCPU 和 8 GiB 内存。

表 6.2: KVM 正确性结果

编号	内容	结果
R01	cryptsetup open/read/close	PASS
R02	dm-crypt 受保护 rootfs	PASS
R03 / V01	dm-verity 正常 rootfs 与正确 root hash	PASS
C01–C05	正确密钥、错误密钥、4/32/256 KiB 往返	PASS
V02	错误 root hash	PASS
V03–V07	首/中/末数据块、叶子/中间 hash block 损坏	PASS

V03–V07 均保持原始正确 root hash，只修改清单指定的物理字节。损坏数据没有成功交付，没有 fallback 到 lower device，也没有以 kernel panic 代替可控 I/O 错误。物理位置见 `PHYSICAL_MUTATION_MANIFEST.md`。

6.3 性能测试结果

表 6.3: 正式性能矩阵完成情况

范围	轮数/单元格	结果
Asterinas FIO dm-crypt	13 项 × 5	PASS
Asterinas FIO dm-verity	7 项 × 5	PASS；6 项 N/A
Linux FIO raw/dm-crypt	26 项 × 5	PASS
Filebench raw/dm-crypt	5 + 5	PASS
SQLite 三目标	5 + 5 + 5	PASS
Asterinas 三场景启动	5 + 5 + 5	PASS

旧 Asterinas dm-crypt FIO 因 mapper 不存在而测试普通文件，已分类为 `HARNESS_BUG` 并从正式输入移除。Linux 与 Asterinas 结果分平台展示，避免把环境差异解释为产品倍率。

6.4 复现命令

```
cargo test --workspace
sudo bash scripts/verify_linux_xts_compat.sh
sudo bash scripts/run_fio_matrix.sh --platform linux-host --runs 5
bash scripts/run_fio_matrix.sh --platform asterinas-kvm \
```

```
  --asterinas-dir "$PWD/asterinas" --runs 5
sudo RUNS=5 bash scripts/run_filebench_matrix.sh
sudo RUNS=5 ROWS=10000 bash scripts/run_sqlite_matrix.sh
RUNS=5 ENABLE_KVM=1 bash scripts/run_boot_matrix.sh
python3 scripts/check_benchmark_package.py
python3 scripts/verify_kvm_evidence.py
python3 scripts/review_release_check.py
```

R01–R03、C01–C05 和 V01–V07 的完整环境变量与命令见仓库根目录 README。

6.5 证据可复核性

- 正式性能结果位于 `docs/evidence/final-performance-20260811/`；
- 正确性日志位于 `docs/evidence/stabilization/correctness-kvm/`；
- kernel build 日志位于 `docs/evidence/stabilization/build-recovery/`；
- 组合栈日志位于 `docs/evidence/stack/`；
- 每条矩阵记录都包含原始证据相对路径，不依赖手工修改数字。

6.6 小结

kernel build、核心 P0 正确性和四类性能测试均有正式 PASS 证据。尚未定义 runner 的扩展 P1 项保持 UNTESTED，不解释为 PASS，也不解释为产品失败。

Chapter 7

合规、来源与 AI 声明

7.1 概述

根据全国操作系统设计赛的技术方案要求，参赛队必须对作品中复制的代码和文档来源进行明确说明，并严格遵守相关开源协议。同时，使用 AI 工具的参赛队需在设计实现文档中声明 AI 工具的名称、使用场景及交互记录。

本章独立成章，全面披露本项目的代码来源、开源协议合规性、借鉴基础与增量贡献，以及 AI 工具的使用情况。所有披露内容均与项目实际仓库、依赖和开发记录一致。

7.2 代码与文档来源及开源协议说明

7.2.1 引用的开源代码

本项目在开发过程中参考或引用了以下开源代码和依赖库：

表 7.1: 引用的开源代码与依赖库

名称	来源/URL	使用目的
星绽操作系统（Asterinas）	`https://github.com/asterinas/asterinas`	基础操作系统框架和 OSTD 接口
`aes` crate	`https://crates.io/crates/aes`	AES 加密算法实现（RustCrypto 项目）
`xts-mode` crate	`https://crates.io/crates/xts-mode`	XTS 加密模式实现（RustCrypto 项目）

7.2.2 开源协议合规性声明

本项目严格遵守以下开源协议要求：

- **项目源代码**（位于 `crates/`、`asterinas/kernel/comps/dm/` 等）：遵循 MIT 开源协议（见仓库根目录 `LICENSE` 文件）。
- **技术文档与答辩材料**（位于 `docs/document/`）：遵循 CC-BY-SA 4.0 开源协议。
- **第三方依赖**：所有引用的 Rust crate（`aes`、`xts-mode` 等）均遵循其各自的开源协议（主要为 MIT 或 Apache-2.0），与本项目的 MIT 协议兼容。
- **星绽操作系统**：作为基础框架，星绽本身采用 MPL-2.0 协议，本项目与之兼容（MIT 与 MPL-2.0 均为宽松许可证，可组合使用）。

7.3 借鉴基础版本与增量贡献说明

7.3.1 借鉴基础

本项目的设计与实现深受以下基础项目和技术的启发：

- **Linux 内核 Device Mapper 框架**：`dm-crypt` 和 `dm-verity` 的功能逻辑、表格式规范（`DmCryptTable`）和 IV 生成语义（plain64）参考了 Linux 内核的成熟设计。
- **星绽操作系统 Framekernel 架构**：基于星绽的 OSTD 框架和 `BlockDevice` 抽象进行适配和集成，具体补丁序列位于 `asterinas_port/patches/`。
- **Rust 生态的密码学库**：借鉴了 `aes` 和 `xts-mode` crate 的接口设计和安全实践，但本项目的 AES-256-XTS 引擎为独立实现，不直接复制其算法代码。

7.3.2 本队增量贡献

在借鉴上述基础版本的基础上，本项目的核心增量贡献包括：

- **统一的 `BlockDevice` Trait 抽象**：设计了与平台无关的块设备抽象层，使得同一套核心代码能够同时运行于用户态（`dmctl`）和星绽内核态（`aster-dm`）。
- **纯 Rust 实现的 SHA-256**：独立实现了 FIPS 180-4 标准的 SHA-256 哈希算法（位于 `crates/dm/src/sha256.rs` 和内核 `asterinas/kernel/comps/dm/src/sha256.rs`），无外部依赖。
- **星绽内核完整集成**:通过可审计 Git 补丁序列（`asterinas_port/patches/0001--0006`）将 `dm-crypt` 和 `dm-verity` 完整集成到星绽内核中，包括启动参数解析、BIO 路径适配、根文件系统切换。
- **内核启动自检机制**:实现了基于 ext2 超级块魔数（`0x53EF`）的启动时自检（`self_test_ext2_magi`）验证加解密路径正确性（补丁 0002）。
- **五阶段渐进式保护路径**：设计并实现了从 Plain 到 Combined（dm-crypt + dm-verity 叠加）的渐进式安全保护路径。

- **性能设计**：实现了认证后哈希块缓存、完成 BIO 数据复用、有界临时缓冲池（scratch pool）和 AES-NI 硬件加速；正式数字仅引用最终真实性审计后的矩阵。
- **Linux 兼容控制面**：实现了 `/dev/mapper/control` 的 Linux `dm_ioctl` 兼容接口，支持 `cryptsetup` 动态创建/删除映射器。

7.4 AI 工具使用声明

本项目在开发过程中，严格遵守比赛关于“允许参赛学生合理使用 AI 工具”的规定。

7.4.1 使用的 AI 工具及大模型

表 7.2: AI 工具使用详情

工具名称	相关大模型	具体使用场景与辅助内容
OpenAI Codex	GPT-5	1. 阅读和梳理星绽框架、`dm-crypt`、`dm-verity` 相关底层代码； 2. 辅助分析 BIO 数据路径与编译错误； 3. 生成或修改部分 Rust、Shell 测试和构建脚本； 4. 协助整理参赛文档表述。
通义千问（Qwen）	Qwen-Max	技术文档的深度润色，确保中英文专业术语的准确性与一致性。
豆包（Doubao）	豆包大模型	答辩表述的逻辑优化，整理技术答辩常见问题及应对策略。
WorkBuddy	腾讯混元 Hy3、DeepSeek	辅助执行测试、完整内核构建和 QEMU 验证并解读日志。

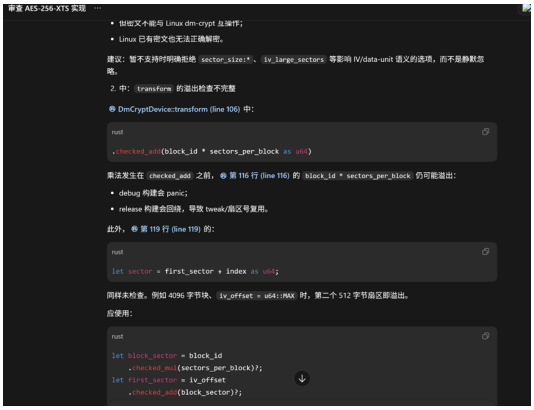
7.4.2 AI 交互记录

以下是开发过程中与 AI 工具的关键交互截图，证明 AI 仅作为辅助工具，开发逻辑由团队主导。

OpenAI Codex 交互记录（GPT-5）

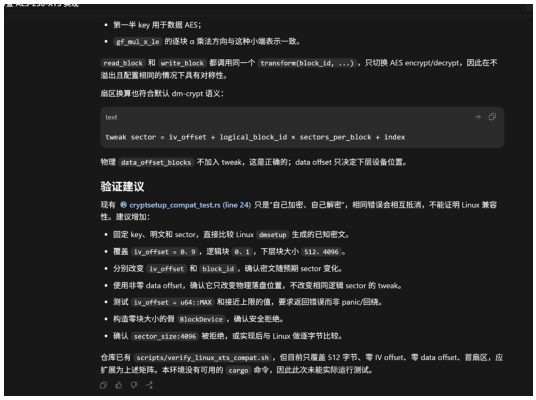


(a) 代码分析与框架理解



(b) BIO 路径辅助分析

图 7.1: OpenAI Codex 交互记录（1/4）



(a) Rust 测试脚本生成



(b) Shell 构建脚本辅助

图 7.2: OpenAI Codex 交互记录（2/4）

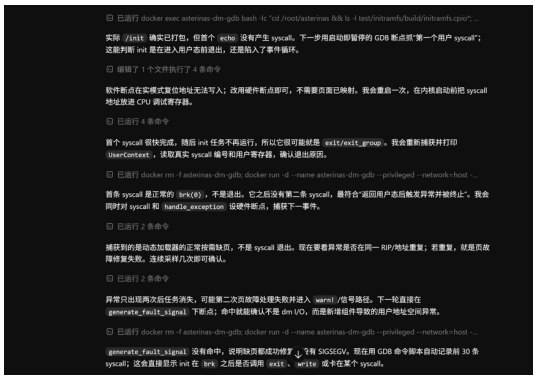


(a) QEMU 日志解读



(b) 编译错误调试

图 7.3: OpenAI Codex 交互记录（3/4）



(a) 内核 panic 追踪



(b) 文档表述整理

图 7.4: OpenAI Codex 交互记录（4/4）

通义千问（Qwen）交互记录



(a) 技术文档润色



(b) 专业术语校对

图 7.5: 通义千问（Qwen）交互记录

豆包（Doubao）交互记录



图 7.6: 豆包（Doubao）答辩表述优化与 FAQ 整理

7.4.3 验证机制与责任声明

为确保代码与文档的质量及原创性，项目组采取了以下严格措施：

1. **人工审查**：所有 AI 生成的代码片段和文档内容，均经过团队成员的人工 Peer Review。
2. **交叉验证**：代码正确性通过严格的单元测试、与 Linux dm-crypt 的逐字节对照、星绽内核构建及 QEMU 端到端运行进行了多重验证。
3. **责任承担**：最终的技术方案、代码取舍、架构决策和提交材料均由参赛团队全权负责。

7.5 Git 提交与开发迭代记录

7.5.1 提交概览

以下为项目主要 Git 提交记录（基于主仓库历史）：

表 7.3: Git 提交记录概览

序号	提交时间	Commit Hash	提交说明	变更统计
1	06-23 14:15	6bb9bc9	初始提交，仅源码	44 文件，+2500 行
2	06-23 22:36	40a5217	Linux 兼容 Device Mapper 实现	18 文件，+1923/-61 行
3	06-23 22:58	c72462b	验证 Asterinas dm BIO 路径	6 文件，+280/-12 行
4	06-24 01:33	2f8a671	接入受保护根文件系统	12 文件，+186/-25 行
5	06-24 01:42	a0dbb0f	补充 AI 工具使用说明	1 文件，+4 行
6	06-24 01:48	84970aa	文档中文化	6 文件，+118/-113 行
7	06-29 21:06	f587893	添加 LaTeX 技术文档、PDF 与答辩 PPT	多文件
8	06-30 17:08	c22b8b4	添加演示视频入口	1 文件
9	07-14 18:44	5f345a1	完成安全存储性能对照与验收证据	多文件

此外，星绽内核集成补丁（asterinas_port/patches/）包含更细粒度的演进历史：

- 0001：组件集成与目标定义
- 0002：BIO 路径验证与自检（EXT2 魔数）
- 0003：根文件系统 pivot_root 切换
- 0004：Runtime 控制面与 cryptsetup 兼容
- 0005：性能优化（缓存、缓冲池、基准）
- 0006：性能阶段归因与正式合同

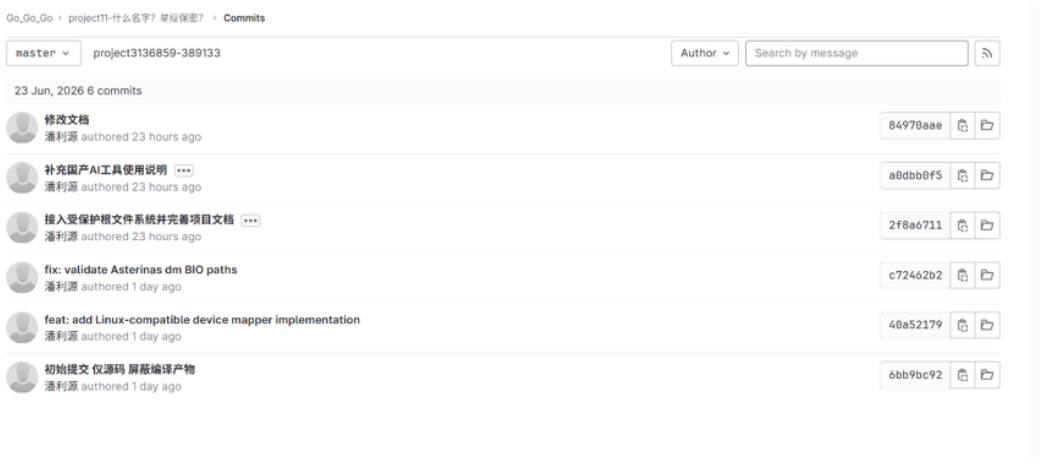


图 7.7: GitLab 仓库提交历史截图

7.5.2 关键迭代节点说明

1. **基础架构与核心逻辑**（Commit `40a5217`）：完成了 `BlockDevice` trait 的抽象，实现了纯 Rust 的 AES-256-XTS 加密与 SHA-256 默克尔树校验。
2. **内核集成与 BIO 路径打通**（Commit `c72462b`、`2f8a671`）：将用户态逻辑移植至星绽内核，成功实现了基于 `pivot_root` 的受保护根文件系统切换。
3. **性能优化与验收证据**（Commit `5f345a1`）：完成安全存储性能对照，包括正式 256 MiB 五轮基准、Linux guest 同条件对照、启动时间测量和哈希缓存 A/B 验证。
4. **文档与合规性完善**（Commit `84970aa`、`f587893`）：全面梳理技术文档，完成中文化翻译，补充 AI 工具使用声明，并添加 LaTeX 技术文档和答辩 PPT。

7.6 小结

本章严格按照全国操作系统设计赛的技术方案要求，全面披露了本项目的代码来源、开源协议合规性、借鉴基础与增量贡献、AI 工具使用声明以及 Git 提交与开发迭代记录。所有引用和借鉴的内容均已明确标注来源，所有 AI 辅助生成的内容均经过人工审查和验证。

Chapter 8

用户手册与部署指南

8.1 概述

本章为开发者提供完整的部署指南，帮助用户从零开始搭建开发环境、编译星绽内核、配置启动参数并验证 dm-crypt 与 dm-verity 的功能。所有步骤均已在 Ubuntu 24.04 LTS 上验证通过。

8.2 运行环境准备

8.2.1 宿主机要求

推荐使用以下环境进行开发和测试：

- **操作系统**：Ubuntu 24.04 LTS（或 Windows WSL2 上的 Ubuntu 24.04）
- **架构**：x86_64
- **内存**：至少 8 GiB
- **磁盘空间**：至少 20 GiB
- **CPU**：支持硬件虚拟化（VT-x/AMD-V）和 AES-NI（推荐）

8.2.2 依赖工具安装

表 8.1: 开发环境依赖

依赖项	版本要求	用途
Rust / Cargo	1.75.0+	核心语言工具链
QEMU	8.2.2+	硬件虚拟化
Docker	29.4.3+	容器化构建环境（仅用于内核构建）
星绽镜像	`asterinas/asterinas:0.17.0-20260618`	官方工具链
Clang / LLVM	18.0.0+	内核编译

Ubuntu 24.04 上可通过以下命令安装基础依赖：

```
sudo apt update
sudo apt install -y build-essential clang lld qemu-system-x86 docker.io
```

Rust 工具链安装：

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
rustup default stable
```

8.3 快速开始：用户态演示

本节演示如何在宿主机上直接运行 `dmctl` 工具，无需启动虚拟机即可验证核心功能。

8.3.1 克隆仓库与运行测试套件

```
git clone <repository-url>
cd OS-
cargo test
```

预期输出：所有测试通过（当前 13+ 个测试，包括 `crypt_test`、`verity_test`、`file_device_test`、`cryptsetup_compat_test` 等）。

8.3.2 演示透明加密（dm-crypt）

```
cargo run -p dmctl -- demo-crypt
```

预期输出：

- plain: 显示可读明文
- raw-hex: 显示十六进制密文，与明文不同
- read: 显示恢复的明文，与原始明文一致
- ciphertext_differs: true

8.3.3 演示完整性校验（dm-verity）

```
cargo run -p dmctl -- demo-verity
```

预期输出：

- 输出 root-hash: <64-char-hex>
- 篡改后读取报告 tamper_detected: true block=1

8.3.4 演示组合栈（dm-crypt + dm-verity）

用户态：

```
cargo run -p dmctl -- demo-stack
```

预期输出：显示两层映射器（crypt-root、verified-root）及其安全能力（confidentiality、integrity、read_only），并成功读取数据。

内核 Combined 叠栈（补丁 0007）：

```
sudo bash scripts/build_stack_rootfs.sh
ENABLE_KVM=1 bash scripts/run_dm_stack.sh
```

关键 cmdline:dm_stack=crypt,verity、dm_crypt.device=vdc、rootfs_verity.device=dm-cr
dm-root /dev/dm-verity0。预期串口含 [dm] stacked mapper: vdc -> dm-crypt0
-> dm-verity0 以及 PASS system root switched to protected mapper。

8.3.5 手动命令示例

初始化与原始 I/O

创建测试镜像（8 块，每块 64 字节）：

```
cargo run -p dmctl -- init-image target/demo.img 8 64
```

直接读取底层（绕过映射器）：

```
cargo run -p dmctl -- raw-read-hex target/demo.img 64 1
```

篡改指定块内的字节（测试 verity）：

```
cargo run -p dmctl -- tamper target/demo.img 64 1 0 ff
```

加密命令（dm-crypt）

加密写入：

```
cargo run -p dmctl -- crypt-write target/demo.img 64 my-secret-key 1 "hello dm-cryp
```

通过映射器解密读取：

```
cargo run -p dmctl -- crypt-read target/demo.img 64 my-secret-key 1
```

使用 Linux 兼容表格式填充密文（用于交叉验证）：

```
cargo run -p dmctl -- crypt-table-fill target/demo.img 512 \
  "aes-xts-plain64 <64-byte-hex-key> 0 ignored 0" 0 5a 1
```

完整性命令（dm-verity）

生成根哈希：

```
root=$(cargo run -p dmctl -- verity-root target/demo.img 64)
```

验证完整性：

```
cargo run -p dmctl -- verity-verify target/demo.img 64 "$root"
```

为星绽内核格式化镜像（追加 on-disk 哈希树）：

```
cargo run -p dmctl -- verity-format-dm target/demo.img 1024
```

微基准测试

```
cargo run -p dmctl -- benchmark 4096 5
```

输出 CSV 格式的吞吐和延迟数据，对比 plain / AES-XTS / verity 三种配置。

8.4 星绽内核集成

8.4.1 准备 Asterinas 源码

在宿主机上拉取星绽源码并切换到基线 commit：

```
git clone https://github.com/asterinas/asterinas.git
cd asterinas
git checkout c6284a9106f5a4c87deb7c8a990af6211dc5f540
```

8.4.2 应用 Device Mapper 补丁

将本项目中的补丁序列应用到 Asterinas 源码：

```
git am /path/to/OS-/asterinas_port/patches/0001-feat-dm-integrate-crypt-and-verity-
       /path/to/OS-/asterinas_port/patches/0002-fix-dm-validate-crypt-and-verity-BI
       /path/to/OS-/asterinas_port/patches/0003-rootfs-switch.patch \
       /path/to/OS-/asterinas_port/patches/0004-Rust-Device-Mapper.patch \
       /path/to/OS-/asterinas_port/patches/0005-dm.patch \
       /path/to/OS-/asterinas_port/patches/0006-dm-performance-attribution.patch
```

补丁说明：

- 0001：组件集成与目标定义（`aster-dm` 注册、启动参数）
- 0002：BIO 路径验证与自检（EXT2 魔数 `0x53EF`）
- 0003：根文件系统 `pivot_root` 切换
- 0004：Runtime 控制面与 `cryptsetup` 兼容
- 0005：性能优化（hash cache、scratch pool、基准）
- 0006：性能阶段归因与正式合同

8.4.3 构建内核

使用官方 Docker 容器构建（推荐）：

```
docker run --rm --privileged --network=host \
  -v asterinas-src:/root/asterinas \
  -w /root/asterinas \
  asterinas/asterinas:0.18.0-20260618 make kernel
```

启用 AES-NI 硬件加速构建：

```
docker run --rm --privileged --network=host \
  -v asterinas-src:/root/asterinas \
  -w /root/asterinas \
  -e OSDK_ENABLE_AES_NI=1 \
  asterinas/asterinas:0.18.0-20260618 make kernel
```

构建产物位于 `/root/asterinas/target/osdk/aster-kernel-osdk-bin.iso`。

8.4.4 构建受保护的根文件系统镜像

在项目根目录运行：

dm-crypt 加密镜像：

```
sudo bash scripts/build_crypt_rootfs.sh
```

镜像生成于 target/crypt-rootfs.img,并附带环境变量文件 target/crypt-rootfs.img.env
包含密钥和配置参数。

dm-verity 校验镜像：

```
bash scripts/build_verity_rootfs.sh
```

镜像生成于 target/verity-rootfs.img,并附带环境变量文件 target/verity-rootfs.img.e
包含根哈希、数据块数、哈希起始块号等。

8.5 系统运行与参数配置

8.5.1 QEMU 启动命令

基本启动命令（使用官方内核 ISO）：

```
qemu-system-x86_64 \
  -accel kvm -cpu host -smp 1 -m 8G \
  -machine q35,kernel-irqchip=split \
  -cdrom target/osdk/aster-kernel-osdk-bin.iso \
  -drive file=target/verity-rootfs.img,format=raw,if=virtio \
  -nographic -display none
```

对于已应用补丁的内核，使用 asterinas/ 目录下的启动脚本更方便：

```
cd asterinas
DM_IMAGE=../target/verity-rootfs.img ENABLE_KVM=1 \
  bash tools/run_dm_verity.sh
```

8.5.2 内核启动参数配置

启用 dm-verity（保护根文件系统完整性）：

```
rootfs_verity.scheme=dm-verity
rootfs_verity.device=vdc
rootfs_verity.name=dm-verity0
rootfs_verity.hash=6db653b69a094065756dc2770d3af8880ec9beed29df41b5bb2fb2e11b2d49f3
rootfs_verity.data_blocks=65536
rootfs_verity.hash_start=65536
rootfs_verity.cache_blocks=256
```

启用 dm-crypt（加密根文件系统）：

```
dm_crypt.device=/dev/vda
dm_crypt.name=dm-crypt0
dm_crypt.key=000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f202122
dm_crypt.scratch_pool=32
```

启用性能阶段统计（仅基准测试）：

```
dm.perf_stats=1
```

启动流程的关键阶段（如图 8.1 所示）：

1. QEMU 加载星绽内核和 initramfs。
2. 内核解析命令行参数，构建配置。
3. `aster-dm` 组件初始化映射器链（`VirtioBlkAdapter` → `DmCryptDevice` / `DmVerityDevice`）。
4. 执行 `self_test_ext2_magic` 自检（检查 EXT2 魔数 `0x53EF`），失败则 panic。
5. initramfs 使用 `dm-root` 模式执行 `pivot_root` 切换到受保护根文件系统。
6. 系统进入用户态，串口输出 `PASS system root switched to protected mapper`。

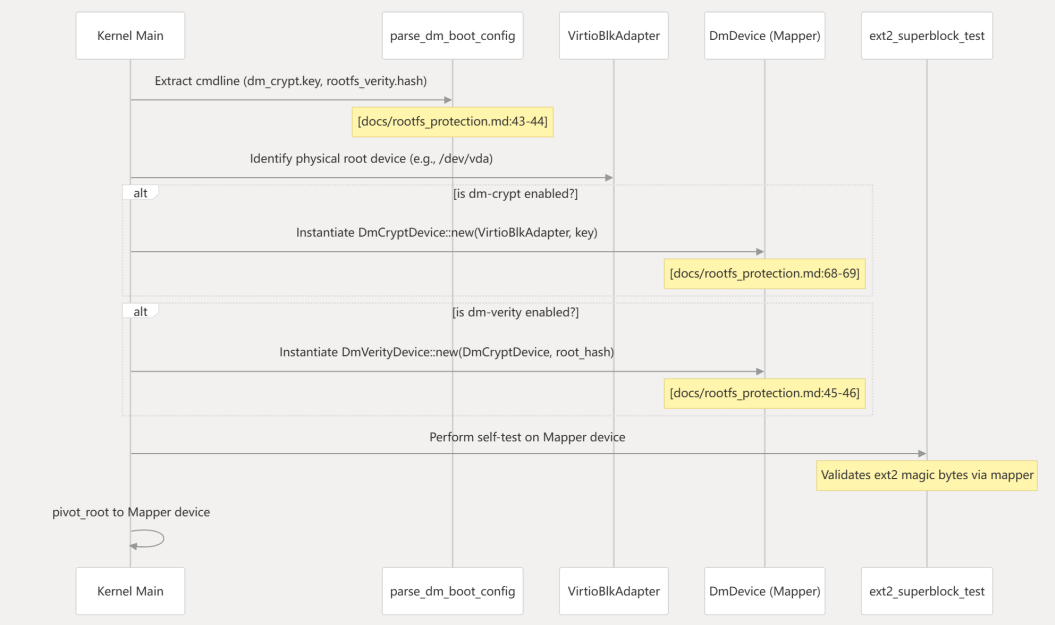


图 8.1: 根保护初始化序列图

8.6 基准测试与性能验证

8.6.1 正式性能测试

Linux host 与 Asterinas KVM 分平台执行，每个工况至少五轮：

```
sudo bash scripts/run_fio_matrix.sh --platform linux-host --runs 5
bash scripts/run_fio_matrix.sh --platform asterinas-kvm \
  --asterinas-dir "$PWD/asterinas" --runs 5
sudo RUNS=5 bash scripts/run_filebench_matrix.sh
sudo RUNS=5 ROWS=10000 bash scripts/run_sqlite_matrix.sh
```

正式输出位于 `docs/evidence/final-performance-20260811/`。旧假 mapper、失败重试和调试结果不会进入生成器输入。

8.6.2 启动时间测量

执行五轮交替冷启动测试：

```
sudo bash scripts/build_asterinas_boot_images.sh
RUNS=5 ENABLE_KVM=1 bash scripts/run_boot_matrix.sh
```

8.6.3 用户态微基准测试

```
bash scripts/benchmark.sh
```

输出 CSV 文件包含 plain / AES-XTS / verity 三种配置的吞吐量对比。

8.7 高级配置与调优

8.7.1 dm-crypt 临时缓冲池调优

启动参数 `dm_crypt.scratch_pool` 控制临时缓冲区池大小（默认 32）。设为 0 可关闭池，便于 A/B 性能对照：

```
dm_crypt.scratch_pool=0
```

8.7.2 dm-verity 哈希缓存调优

启动参数 `rootfs_verity.cache_blocks` 控制已认证哈希块缓存大小（默认 256）。增大容量可提高缓存命中率，但增加内存占用：

```
rootfs_verity.cache_blocks=512
```

8.7.3 内核阶段统计

开启阶段统计（用于性能归因）：

```
dm.perf_stats=1
```

统计信息在每轮 fio 前后通过 `/proc/dm-performance-stats` 输出，包含以下阶段：

- `validation_ns`、`scratch_ns`、`copy_ns`、`crypto_ns`、`lower_wait_ns`（dm-crypt）
- `data_sha256_ns`、`cache_lookup_ns`、`hash_wait_ns`、`hash_sha256_ns`、`digest_compare_ns`、`cache_insert_ns`（dm-verity）

8.8 故障排查

8.8.1 常见问题

- **内核构建失败**：确保 Docker 容器使用正确的 Asterinas commit（`c6284a9`）并已应用全部 6 个补丁。检查 `asterinas_port/README.md` 中的补丁顺序。
- **QEMU 无法启动**：检查是否安装了 KVM 支持（`/dev/kvm` 是否存在），并确保镜像路径正确。对于 WSL2 环境，需要确认 `/dev/kvm` 可用。
- **dm-verity 启动失败**：确认根哈希与镜像匹配。使用 `dmctl verity-root` 重新生成根哈希，并与启动参数中的 `rootfs_verity.hash` 对比。检查 `data_blocks` 和 `hash_start` 参数是否与镜像一致。
- **dm-crypt 解密失败**：检查密钥是否正确（64 字节十六进制），且 `iv_offset` 和 `offset` 参数与加密时一致。查看串口日志确认 `backend` 输出（AES-NI 或 portable-software）。
- **性能数据不可复现**：确保使用 `release-lto` 构建，KVM 加速和 `CPU=host`，且 fio 合同完全一致（2s ramp + 10s runtime，交替五轮）。检查 `dm.perf_stats=1` 是否开启。
- **自检失败（EXT2 魔数不匹配）**：确认底层镜像确实是 ext2 格式，且文件系统位于镜像开头（无额外偏移）。对于 dm-crypt，确认 `offset` 参数为 0 或与镜像布局匹配。
- `cryptsetup` 无法控制：检查 `/dev/mapper/control` 是否存在，以及是否有权限访问。在 Asterinas 中，该设备由 `aster-dm` 组件在初始化时注册。

8.8.2 调试技巧

- 查看串口日志:在 QEMU 启动时使用 `-serial stdio` 或 `-serial file:serial.log`。
- 启用内核调试输出:启动参数中添加 `console=ttyS0` 并确保 initramfs 使用 `/dev/console`。
- 阶段统计验证：使用 `dm.perf_stats=1` 并检查 `/proc/dm-performance-stats`。

- 验证镜像完整性：使用 `dmctl verity-verify` 在用户态测试镜像。
- 检查补丁状态：在 Asterinas 源码中运行 `git log --oneline` 确认补丁已应用。

8.9 小结

本章提供了完整的用户手册与部署指南，涵盖了从环境准备、代码编译、镜像构建到系统运行的全流程，并给出了基准测试、调优建议和故障排查方法。通过遵循本章的步骤，开发者可以快速验证核心功能并部署星绽内核到 QEMU/KVM 环境中。

Chapter 9

研发过程与工程收尾

9.1 研发原则

项目采用“先保证安全语义，再接入真实块路径，最后固化复现入口”的顺序：

1. 在用户态用 `BlockDevice` 抽象验证 AES-XTS、Merkle tree 与错误边界；
2. 通过 `VirtioBlkAdapter` 接入 Asterinas BIO，不绕过真实下层设备；
3. 分别完成 crypt、verity 与组合栈，再接入受保护 rootfs；
4. 使用 Linux 兼容性、KVM 正反例和物理损坏测试作为合入门禁；
5. 性能数据必须通过 target、参数、parser 与完成语义审计。

9.2 关键工程问题

9.2.1 块大小与扇区语义

上层文件系统通常以 4 KiB 块发起 I/O，而 dm-crypt plain64 IV 以 512 字节扇区号为单位。实现将每个块拆分为连续扇区，并同时考虑 IV offset 与 data offset；所有加减法都进行越界检查，避免整数溢出改变物理位置。

9.2.2 BIO 生命周期

mapper 只在 lower BIO 完成后完成 upper BIO。读路径先获得密文，再按扇区解密并写回上层 segment；写路径复制明文、加密临时缓冲并提交 lower BIO。错误状态原样传播，不会把失败转换为成功，也不会 fallback 到未保护设备。

9.2.3 verity 信任边界

root hash 由启动参数提供，镜像内数据和 hash tree 均视为不可信。认证后 hash cache 只有在完整路径验证成功后才插入，数据块、叶子 hash block 或中间 hash block

的单字节变化都必须在交付前被拒绝。

9.3 Asterinas 补丁演进

表 9.1: 内核移植补丁职责

补丁	职责
0001	Device Mapper 组件骨架、crypt/verity target 与 VirtIO 适配
0002	启动参数、mapper 注册和 ext2 魔数自检
0003	initramfs 挂载与 `pivot_root`
0004	cryptsetup 控制面与 mapper 生命周期
0005	正式 benchmark 参数和内核统计接口
0006	性能阶段计数与可复核元数据
0007	`dm-crypt → dm-verity` Combined 组合栈

当前仓库 gitlink 已包含正式集成版本；`asterinas_port/patches` 用于说明移植边界与复核历史，不要求评委在正常 checkout 后重复应用。

9.4 渐进式保护路径

表 9.2: 五阶段验证路径

阶段	名称	数据流
1	Plain	virtio-blk → ext2
2	Passthrough	virtio-blk → PassthroughDevice → ext2
3	dm-verity	virtio-blk → DmVerityDevice → ext2
4	dm-crypt	virtio-blk → DmCryptDevice → ext2
5	Combined	virtio-blk → DmCryptDevice → DmVerityDevice → ext2

Combined 的镜像构建与启动入口为 `scripts/build_stack_rootfs.sh` 和 `scripts/run_dm_st`
用户态 `dmctl demo-stack` 使用相同装饰器顺序。

9.5 测试代码收口

最终版本只保留当前正式工作流使用的构建、正确性和性能 runner。一次性调试脚本、旧跨 guest 比较器、失败重试和早期小矩阵不进入最终目录。发布检查执行以下门禁：

```
cargo test --workspace
python3 scripts/check_benchmark_package.py
python3 scripts/verify_kvm_evidence.py
python3 scripts/review_release_check.py
```

静态审查要求 Python 不存在未使用函数/import，shell 函数必须存在实际调用；生成镜像和缓存位于 `target`，不上传到代码仓库。

9.6 已知边界

- 当前主路径聚焦 AES-256-XTS、plain64 与 SHA-256，不宣称覆盖 Linux 的全部 target 选项；
- 用户态 verity tree 用于快速测试，Asterinas 内核路径使用 on-disk hash tree；
- dm-verity 为只读目标，写入 workload 明确记为 N/A；
- 不同平台的性能环境分别报告，避免把宿主机差异解释为产品收益；
- 密钥注入的生产化方案仍需结合可信启动、远程证明和密钥管理系统。

9.7 小结

研发过程最终收敛为单一正式代码路径、单一性能矩阵生成器和最小可复核证据链。核心安全实现没有为了改善跑分而放宽验证条件，历史无效数据也不会进入最终公开结论。

附录 A

附录 A：术语表

术语	定义	代码位置
BlockDevice	表示任何可以读写固定大小块的存储介质的 trait 抽象	crates/dm/src/block_device.rs
dm-crypt	提供透明磁盘加密的设备映射器目标	crates/dm/src/dm_crypt.rs
dm-verity	提供透明完整性校验的设备映射器目标	crates/dm/src/dm_verity.rs
HashTree	dm-verity 使用的默克尔树结构	crates/dm/src/hash_tree.rs
AES-256-XTS	磁盘加密的标准密码模式	crates/dm/src/crypto.rs
plain64 IV	使用 64 位扇区号作为 IV 的策略	crates/dm/src/crypto.rs
IntegrityViolation	哈希不匹配时返回的错误	crates/dm/src/error.rs
Root Hash	默克尔树的顶层哈希，信任锚	crates/dm/src/hash_tree.rs
Tweak Key	XTS 密钥的后半部分，用于加密 IV	crates/dm/src/crypto.rs
IV Offset	用于计算初始化向量的偏移值	crates/dm/src/dm_crypt.rs
Data Offset	物理设备开头用于元数据的扇区数	crates/dm/src/dm_crypt.rs
DM_SECTOR_SIZE	设备映射器的原子 I/O 单位（512 字节）	crates/dm/src/dm_crypt.rs
DmDevice	星绽内核中实现 BIO 映射的块设备	asterinas/kernel/comps/dm/src/devi
DM 启动参数	crypt/verity 参数解析与 mapper 注册	asterinas/kernel/comps/dm/src/lib
PassthroughDevice	直接转发调用的包装器	crates/dm/src/device_mapper.rs
aster-dm	星绽内核中的 Device Mapper 组件	asterinas/kernel/comps/dm/
DmTarget	封装 Crypt 或 Verity 逻辑的枚举	asterinas/kernel/comps/dm/src/devi
self_test_ext2_magic	启动时自检函数（检查 EXT2 魔数）	补丁 0002
pivot_root	切换根文件系统的机制	补丁 0003
dmctl	用户态命令行工具	crates/dmctl/src/main.rs
MemoryBlockDevice	基于内存的后端	crates/dm/src/block_device.rs
FileBlockDevice	基于文件的后端	crates/dm/src/block_device.rs
VirtioBlkAdapter	星绽内核中 VirtIO 块设备适配器	asterinas/kernel/comps/dm/src/devi

术语	定义	代码位置
Scratch Pool	dm-crypt 有界临时缓冲池，用于复用加解密缓冲区	asterinas/kernel/comps/dm/src/dev
认证后哈希缓存	只缓存经过完整 Merkle 路径认证的哈希块	asterinas/kernel/comps/dm/src/ver
DmStackBuilder	用户态下组合 crypt/verity 层的构建器	crates/dm/src/device_mapper.rs

附录 B

附录 B：dmctl 命令参考

初始化与原始 I/O 命令

命令	签名与示例	实现位置
init-image	**签 名**：init-image <path> <blocks> <block_size> **示例**：dmctl init-image disk.img 1024 4096 **说明**：创建零填充文件用作虚拟块设备	`dmctl/src/main.rs`
raw-read-hex	**签 名**：raw-read-hex <path> <block_size> <block_id> **示 例**：dmctl raw-read-hex disk.img 4096 0 **说明**:读取块以十六进制打印（绕过映射器）	`dmctl/src/main.rs`
tamper	**签 名**：tamper <path> <block_size> <block_id> <offset> <hex_byte> **示例**：dmctl tamper disk.img 4096 0 10 ff **说明**：修改块内字节（测试 verity）	`dmctl/src/main.rs`

加密命令（dm-crypt）

命令	签名与示例	实现位置
crypt-write	签 名：crypt-write <path> <block_size> <key> <block_id> <data_string> 示例：dmctl crypt-write disk.img 4096 mysecretkey 0 ”Hello”	`dmctl/src/main.rs`
crypt-read	签 名：crypt-read <path> <block_size> <key> <block_id> 示例：dmctl crypt-read disk.img 4096 mysecretkey 0	`dmctl/src/main.rs`
crypt-table-fill	签 名：crypt-table-fill <path> <block_size> <table_str> <start> <byte> <count> 示 例：dmctl crypt-table-fill disk.img 4096 ”aes-xts-plain64 <key> 0 /dev/loop0 0” 0 aa 10	`dmctl/src/main.rs`

完整性命令（dm-verity）

命令	签名与示例	实现位置
verity-root	签 名：verity-root <path> <block_size> 示例：dmctl verity-root disk.img 4096	`dmctl/src/main.rs`
verity-verify	签 名：verity-verify <path> <block_size> <root_hash_hex> 示 例：dmctl verity-verify disk.img 4096 a1b2...	`dmctl/src/main.rs`
verity-format-dm	签名：verity-format-dm <path> <data_blocks> 示例：dmctl verity-format-dm disk.img 1024 说明：为星绽内核格式化镜像	`dmctl/src/main.rs`

演示与基准测试命令

命令	签名与示例	实现位置
demo-crypt	签名：demo-crypt 示例：dmctl demo-crypt	`dmctl/src/main.rs`
demo-verity	签名：demo-verity 示例：dmctl demo-verity	`dmctl/src/main.rs`
demo-stack	签名：demo-stack 示例：dmctl demo-stack	`dmctl/src/main.rs`
benchmark	签名：benchmark [blocks] [iterations] 示例：dmctl benchmark 4096 5	`dmctl/src/main.rs`

参考文献

[1] Linux Kernel Community. *Linux dm-crypt Documentation*, 2024. Linux dm-crypt 内核文档.

[2] Linux Kernel Community. *Linux dm-verity Documentation*, 2024. Linux dm-verity 内核文档.

[3] Milan Broz et al. cryptsetup - linux unified key setup (luks). https://gitlab.com/cryptsetup/cryptsetup, 2024. LUKS 和 dm-crypt 用户态工具.

[4] National Institute of Standards and Technology (NIST). Nist special publication 800-38e: Recommendation for block cipher modes of operation: The xts-aes mode for confidentiality on storage devices. https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38e.pdf, 2010. XTS-AES 存储加密模式标准.

[5] National Institute of Standards and Technology (NIST). Fips pub 180-4: Secure hash standard (shs). https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf, 2015. SHA-256 标准规范.

[6] Asterinas Team. 星绽操作系统 (asterinas). https://github.com/asterinas/asterinas, 2024. 访问日期：2026 年 8 月.